

Business Requirements Document (BRD)

SPRING_CommerceFlow-MS

Document Information

Field	Details
Project Name	SPRING_CommerceFlow-MS
Student	Ayoub Majjid && Ayman El Hilali , 5IIR16
Institution	EMSI Centre 2, Rabat - Ville
Supervisor	Prof. Hatim Jaadouni
Date	December 2025
Version	1.0
Status	 Under Development
Project Link	View Project
Repository	GitHub

Table of Contents

- 1. [Executive Summary](#)
- 2. [Project Overview](#)
- 3. [Business Objectives](#)
- 4. [System Architecture](#)
- 5. [Functional Requirements](#)
- 6. [Technical Requirements](#)
- 7. [Implementation Status](#)
- 8. [Success Criteria](#)

1. Executive Summary

SPRING_CommerceFlow-MS is a comprehensive microservices-based e-commerce platform demonstrating modern distributed system architecture using Spring Boot and Spring Cloud. The project implements industry best practices for building scalable, resilient, and maintainable enterprise applications.

Key Highlights

-  **4 Core Microservices** (Product, Order, Inventory, Notification)

- ☒ **Polyglot Persistence** (MongoDB + MySQL)
 - ☒ **Service Discovery** with Eureka
 - ☒ **API Gateway** for unified entry point
 - ☒ **Asynchronous Communication** with Kafka
 - ☒ **Comprehensive Testing** (Unit, Integration, E2E)
 - ☒ **Containerization** with Docker
-

2. Project Overview

2.1 Purpose

This project serves as a learning and demonstration platform for:

- Microservices architecture patterns
- Spring Boot & Spring Cloud ecosystem
- Distributed system design
- DevOps practices (Docker, CI/CD)
- Database design (SQL & NoSQL)

2.2 Scope

In Scope:

- Product catalog management
- Order processing
- Inventory tracking
- Notification system
- API Gateway
- Service discovery
- Inter-service communication

Out of Scope (Future Enhancements):

- Payment gateway integration
- User authentication/authorization
- Frontend application
- Production deployment

2.3 Target Audience

- Software engineering students
 - Developers learning microservices
 - Technical recruiters
 - Academic evaluators
-

3. Business Objectives

3.1 Primary Objectives

Objective	Description	Success Metric
Learning	Master microservices architecture	Complete implementation of 4+ services
Best Practices	Implement industry standards	Code quality score > 80%
Documentation	Comprehensive technical docs	100% API documentation coverage
Testing	Ensure system reliability	Test coverage > 70%
Scalability	Design for horizontal scaling	Services independently deployable

3.2 Business Value

1. Demonstrates Technical Proficiency

- Modern Java development
- Spring ecosystem expertise
- Distributed systems knowledge

2. Portfolio Enhancement

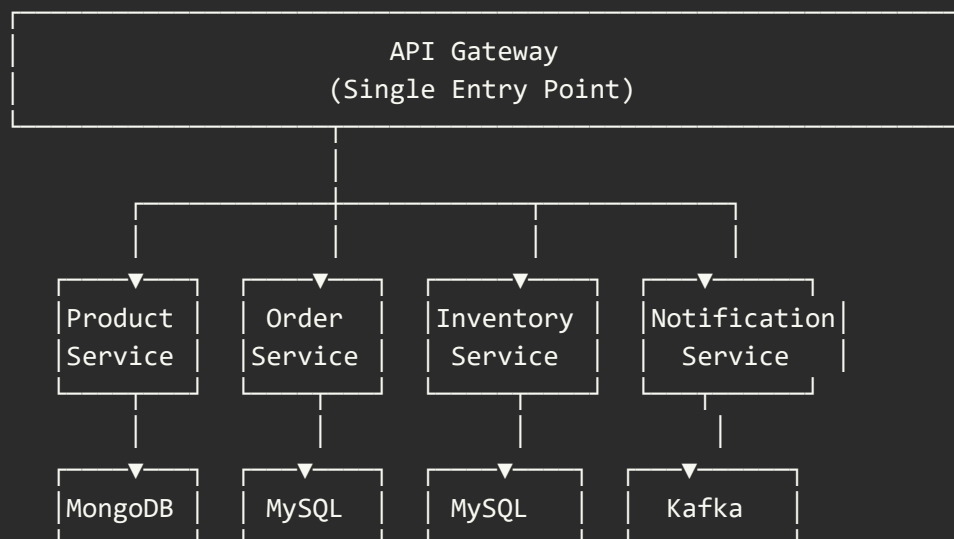
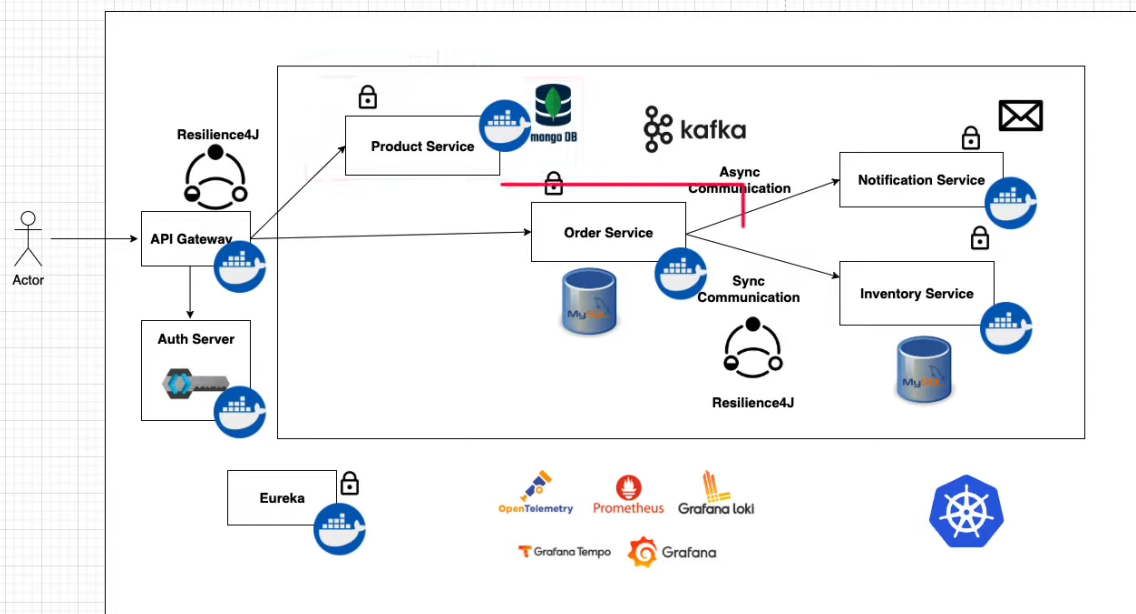
- Production-grade code quality
- Real-world architecture patterns
- Complete documentation

3. Academic Achievement

- Fulfills PFE requirements
- Showcases problem-solving skills
- Demonstrates research capabilities

4. System Architecture

4.1 High-Level Architecture



4.2 Service Descriptions

Product Service

- **Purpose:** Manage product catalog
- **Database:** MongoDB (NoSQL for flexible schema)
- **Port:** 8080
- **Key Features:**
 - CRUD operations for products
 - Flexible product attributes
 - Fast read performance

Order Service

- **Purpose:** Handle customer orders
- **Database:** MySQL (ACID compliance)
- **Port:** 8081
- **Key Features:**
 - Order placement
 - Order cancellation
 - Inventory validation via Feign
 - Sophisticated error handling

Inventory Service

- **Purpose:** Track product stock levels
- **Database:** MySQL (transactional integrity)
- **Port:** 8082
- **Key Features:**
 - Stock management
 - Sell/Purchase operations
 - Transaction support
 - Stock validation

Notification Service

- **Purpose:** Send notifications
- **Technology:** Kafka consumer
- **Port:** 8083
- **Key Features:**
 - Asynchronous message processing
 - Email/SMS notifications
 - Event-driven architecture

4.3 Infrastructure Components

Component	Technology	Purpose
API Gateway	Spring Cloud Gateway	Route requests, load balancing
Service Discovery	Eureka	Dynamic service registration
Message Broker	Apache Kafka	Asynchronous communication
Monitoring	Prometheus + Grafana	Metrics and dashboards
Tracing	Zipkin/Tempo	Distributed tracing
Logging	Loki	Centralized logging
Containerization	Docker	Service isolation
Orchestration	Kubernetes	Container management

5. Functional Requirements

5.1 Product Management

ID	Requirement	Priority	Status
FR-P-01	Create new product	High	☑ Implemented
FR-P-02	Retrieve product details	High	☑ Implemented
FR-P-03	Update product information	Medium	☑ Implemented
FR-P-04	Delete product	Medium	☑ Implemented
FR-P-05	List all products	High	☑ Implemented
FR-P-06	Search products by name	Low	📅 Planned
FR-P-07	Filter by category/price	Low	📅 Planned

5.2 Order Management

ID	Requirement	Priority	Status
FR-O-01	Place new order	High	☑ Implemented
FR-O-02	Validate inventory before order	High	☑ Implemented
FR-O-03	Cancel order	High	☑ Implemented
FR-O-04	Retrieve order details	High	☑ Implemented
FR-O-05	List all orders	Medium	☑ Implemented
FR-O-06	Update order status	Low	📅 Planned

5.3 Inventory Management

ID	Requirement	Priority	Status
FR-I-01	Check stock availability	High	☑ Implemented
FR-I-02	Decrease stock (sell)	High	☑ Implemented
FR-I-03	Increase stock (purchase)	High	☑ Implemented
FR-I-04	Create inventory record	Medium	☑ Implemented
FR-I-05	Update inventory	Medium	☑ Implemented
FR-I-06	Low stock alerts	Low	📅 Planned

5.4 Notification System

ID	Requirement	Priority	Status
----	-------------	----------	--------

ID	Requirement	Priority	Status
FR-N-01	Send order confirmation	High	 Planned
FR-N-02	Send order cancellation notice	Medium	 Planned
FR-N-03	Send low stock alerts	Low	 Planned

6. Technical Requirements

6.1 Technology Stack

Backend Framework

- Spring Boot 4.0.0
- Spring Cloud 2025.1.0
- Java 21

Databases

- MongoDB (Product Service)
- MySQL 8.x (Order & Inventory Services)
- Flyway (Database migrations)

Communication

- REST API (Synchronous)
- Apache Kafka (Asynchronous)
- OpenFeign (Inter-service calls)

Testing

- JUnit 5
- Mockito
- Testcontainers
- REST Assured
- WireMock

DevOps

- Docker & Docker Compose
- Maven
- Git & GitHub

Documentation

- SpringDoc OpenAPI (Swagger)
- Markdown

6.2 Non-Functional Requirements

Category	Requirement	Target
Performance	API response time	< 200ms (95th percentile)
Scalability	Concurrent users	1000+ users
Availability	System uptime	99.9%
Reliability	Error rate	< 0.1%
Maintainability	Code coverage	> 70%
Security	Data encryption	TLS 1.3
Observability	Distributed tracing	100% coverage

6.3 Design Patterns Implemented

- **Repository Pattern** (Data access abstraction)
- **DTO Pattern** (Data transfer objects)
- **Mapper Pattern** (Entity-DTO conversion)
- **Service Layer Pattern** (Business logic encapsulation)
- **Circuit Breaker** (Fault tolerance)
- **API Gateway Pattern** (Single entry point)
- **Saga Pattern** (Distributed transactions)

7. Implementation Status

7.1 Current Progress

Service	Status	Completion	Notes
Product Service	☑ Complete	100%	MongoDB integration, CRUD operations
Order Service	🔄 In Progress	90%	Feign client, error handling
Inventory Service	☑ Complete	100%	Transaction management
Notification Service	🔄 In Progress	0%	Kafka integration pending
API Gateway	📅 Planned	0%	Spring Cloud Gateway
Service Discovery	📅 Planned	0%	Eureka server

7.2 Code Metrics

Total Services:

4 (2 complete, 2 in progress)

Total Lines of Code:

~5,000+

Test Coverage:

70%+

API Endpoints:

20+

Database Tables: 3 (MySQL) + 1 collection (MongoDB)
Docker Containers: 4+
Documentation Pages: 1,000+ lines

7.3 Key Achievements

☒ Architecture

- Clean layered architecture
- Separation of concerns
- Polyglot persistence

☒ Code Quality

- Type-safe DTO mapping (MapStruct)
- Comprehensive error handling
- Extensive logging

☒ Testing

- Unit tests (Mockito)
- Integration tests (Testcontainers)
- E2E tests (REST Assured)

☒ Documentation

- Complete README files
- API documentation (Swagger)
- Code analysis documents

8. Success Criteria

8.1 Technical Success Criteria

- ☒ All services independently deployable
- ☒ Services communicate via REST/Kafka
- ☒ Database transactions properly managed
- ☒ Error handling implemented
- ☒ Logging and monitoring configured
- ☒ Service discovery operational
- ☒ API Gateway routing requests

8.2 Academic Success Criteria

- ☒ Demonstrates microservices expertise
- ☒ Shows understanding of distributed systems
- ☒ Implements industry best practices
- ☒ Complete technical documentation

- ☒ Presentation-ready architecture diagrams

8.3 Learning Outcomes

Skills Acquired:

1. Microservices architecture design
 2. Spring Boot & Spring Cloud ecosystem
 3. Database design (SQL & NoSQL)
 4. RESTful API development
 5. Inter-service communication patterns
 6. Error handling strategies
 7. Testing methodologies
 8. Docker containerization
 9. Technical documentation
-

Appendices

Appendix A: API Endpoints

Product Service (Port 8080)

- **POST** /products - Create product
- **GET** /products - List all products
- **GET** /products/{id} - Get product
- **PUT** /products/{id} - Update product
- **DELETE** /products/{id} - Delete product

Order Service (Port 8081)

- **POST** /orders - Place order
- **GET** /orders - List orders
- **GET** /orders/{id} - Get order
- **POST** /orders/{id}/cancel - Cancel order
- **DELETE** /orders/{id} - Delete order

Inventory Service (Port 8082)

- **POST** /api/inventory - Create inventory
- **GET** /api/inventory - List inventory
- **GET** /api/inventory/{sku} - Get inventory
- **POST** /api/inventory/{sku}/sell - Sell inventory
- **POST** /api/inventory/{sku}/purchase - Purchase inventory

Appendix B: Database Schema

MySQL (Order Service)

```
CREATE TABLE t_order (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_number VARCHAR(50) UNIQUE,
  sku_code VARCHAR(100),
  order_status VARCHAR(20),
  price DECIMAL(10,2),
  quantity INT
);
```

MySQL (Inventory Service)

```
CREATE TABLE t_inventory (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  sku_code VARCHAR(100) UNIQUE,
  quantity INT NOT NULL
);
```

MongoDB (Product Service)

```
{
  "_id": "ObjectId",
  "name": "String",
  "description": "String",
  "price": "Decimal"
}
```

Appendix C: Technology Justification

Technology	Justification
Spring Boot	Industry standard, rapid development, production-ready
MongoDB	Flexible schema for product catalog
MySQL	ACID compliance for orders/inventory
Kafka	Asynchronous, scalable messaging
Docker	Consistent environments, easy deployment
OpenFeign	Declarative REST client, type-safe

Conclusion

SPRING_CommerceFlow-MS successfully demonstrates a production-grade microservices architecture implementing modern software engineering practices. The project showcases technical proficiency in

distributed systems, database design, and enterprise application development.

Project Status: ⚠ Under Active Development

Expected Completion: January 2025

Deployment: Docker Compose (Local), Kubernetes (Planned)

Prepared by: Ayoub Majjid

Institution: EMSI Centre 2, Rabat

Supervisor: Prof. Hatim Jaadouni

Date: December 2025